



UNIVERSITÉ DE MONS

Département d'Informatique

Projet de Réseau

Réalisé par :

Decorte Émile et Tomsen Thomas

222663 et 212281

Groupe 11

Encadrant : Professeur **Valentin Dussolier**

Année académique 2024–2025

Table des matières

1	Exécution	2
2	Implémentation	3
2.1	Structure du code	3
2.2	Détails de l'implémentation	3
2.3	Difficultés rencontrées	4
2.4	Utilisation de l'IA génératrice	4
3	Simulation	5
3.1	Validation de notre implémentation	6
3.1.1	Calcul des tables	6
3.1.2	Lecture des tables	6
3.2	Impact des délais	6
3.3	Comptage à l'infini	7
3.3.1	Explications	8
3.3.2	Source :	9

Chapitre 1

Exécution

Pour exécuter le programme, il faut se placer dans le dossier src et exécuter la commande :

```
python main.py topologies/nom_de_la_topologie
```

Les fichiers Python implémentés (Packet, Link, Router, Network) sont dans le dossier network, les topologies demandées se trouvent dans le dossier topologies.

Chapitre 2

Implémentation

2.1 Structure du code

Dans le dossier network du code nous retrouvons toutes les classes utiles à notre simulateur :

- **Packet** : Cette classe permet de représenter les paquets, nous multiplions la taille des vecteurs envoyés par 32 pour simuler un encodage sur 32 bits.
- **Link** : Elle représente les liens et permet notamment d'induire le délais dû aux vitesses de propagation et de transmission.
- **Router** : Elle contient les fonctions d'envoi/ réception de vecteurs, fait les calculs de meilleur chemin basé sur la méthode à vecteurs de distance ainsi que les logs
- **Network** : Cette classe permet de charger notre topologie (liens et évènements) et d'envoyer les vecteurs initiaux.

Nous retrouvons dans le dossier topologies les différentes topologies demandées dans l'énoncé :

2.2 Détails de l'implémentation

Les fonctions implémentées restent simples, le calcul des routes se produit lorsqu'un routeur reçoit un paquet d'un vecteur voisin, dans la fonction "receive" des routeurs. Le meilleur coût d'un chemin ainsi que le prochain "saut" sont réinitialisés à chaque fois qu'un routeur reçoit un paquet et veut vérifier qu'il n'existe aucun meilleur chemin.

La fonction "send" des routeurs vérifie que le vecteur à envoyer est différent du dernier envoyé. Si oui, il est transmis, sinon il est inutile de le

retransmettre, cela permet d'arriver à une stabilisation du système.

Le délai est calculé dans la fonction `transmit` de la classe `Link`, elle calcule ce dernier sur base de la taille des paquets ainsi que de la vitesse de transmission et de propagation.

La fonction `chang_cost` dans la classe `Network` a été ajoutée pour essayer d'alléger un peu la fonction de d'importation de la topologie.

- **count-to-infinity** : La topologie qui permet de simuler le cas de comptage à l'infini.
- **delay** : Celle qui permet de simuler la topologie donnée pour étudier l'impact du délais sur la formation des tables de routage.
- **verif** : Notre topologie présentée plus en détail dans le chapitre "Simulation" du rapport.

Pour simuler la coupure d'un lien, il faut mettre un coût supérieur à 99999 dans le fichier json car nous ne pouvons pas utiliser `float('inf')` directement dedans.

2.3 Difficultés rencontrées

Retarder l'évaluation d'une fonction passée à `"add_event"` a été un problème. Contrairement à `"send"`, qui n'a pas besoin de passer d'arguments, il nous suffisait de retirer les parenthèses pour qu'à l'appel `"event()"` celle-ci soit évaluée. Les méthodes `"receive"` et `"change_cost"`, elles, nécessitent de passer des variables, ce qui implique une évaluation immédiate de la fonction. Pour remédier à cela nous avons utilisé `"lambda"` qui permet "d'envelopper" notre méthode et ainsi retarder son évaluation.

2.4 Utilisation de l'IA génératrice

Nous avons utilisé l'IA génératrice (ChatGPT) notamment pour l'utilisation de `"lambda"`, qui permet de retarder l'évaluation des événements et ainsi la passer à la fonction `add_event` du simulateur. Elle nous a aussi aidé à faire un affichage propre des tables de routage et des logs.

Chapitre 3

Simulation

Notre topologie est représentée par le schéma suivant :

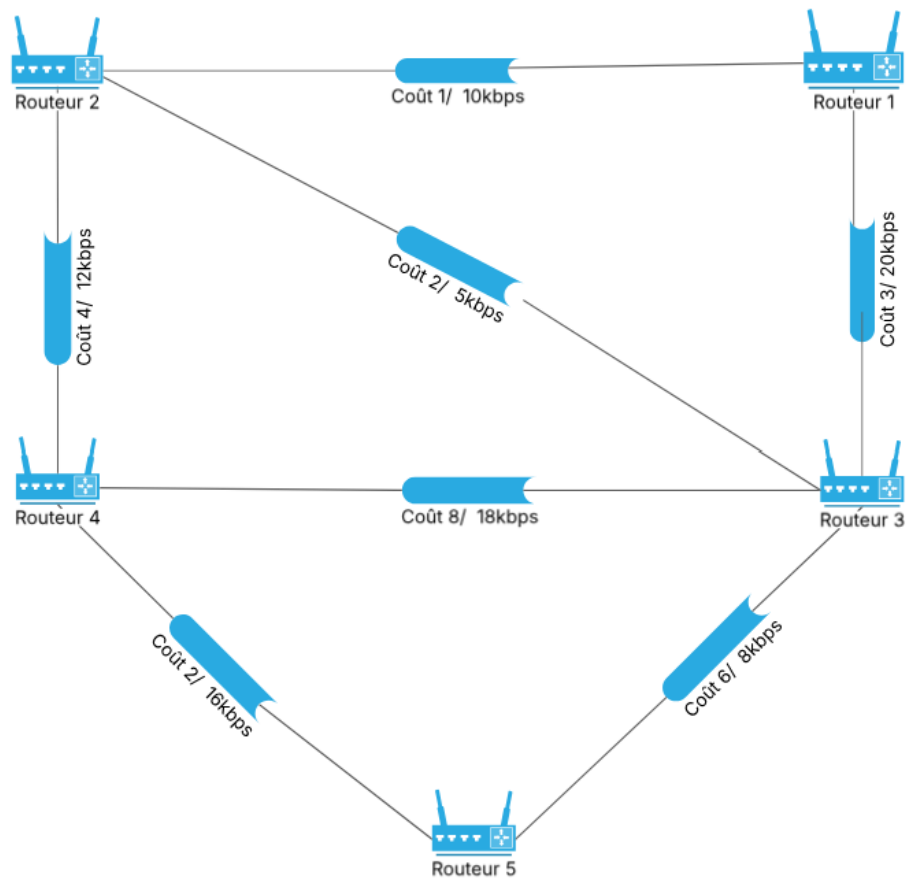


FIGURE 3.1 – Topologie réseau composée de 5 routeurs et 7 liens

3.1 Validation de notre implémentation

3.1.1 Calcul des tables

Pour valider les tables de routage produites par notre implémentation, nous avons exécuté l'algorithme de Bellman-Ford à notre topologie. Nos résultats sont identiques à ceux obtenus par le simulateur.

Les résultats obtenus par l'algorithme sont les suivants :

-- Router 1 :			-- Router 2 :			-- Router 3 :		
dest	cost	next-hop	dest	cost	next-hop	dest	cost	next-hop
2	1	2	1	1	1	1	3	1
3	3	3	3	2	3	2	2	2
4	5	2	4	4	4	4	6	2
5	7	2	5	6	4	5	6	5

-- Router 4 :			-- Router 5 :		
dest	cost	next-hop	dest	cost	next-hop
1	5	2	1	7	4
2	4	2	2	6	4
3	6	2	3	6	3
5	2	5	4	2	4

FIGURE 3.2 – Tables de routages du système à 5 routeurs et 7 liens.

3.1.2 Lecture des tables

Prenons le routeur 1 de la figure 3.2. Le chemin le moins coûteux vers 2 coûte 1 et le prochain routeur vers lequel aller est 2, il faut donc ne faire qu'un "saut", le chemin est direct. Pour aller au routeur 5, le chemin a un coût total de 7, il faut passer par 2. Quand nous regardons à la table du routeur 2, il faut passer par le routeur 4, en regardant à la table de ce dernier, le chemin est direct vers le routeur 5. Si l'on fait la somme des coûts de 1 à 2, 2 à 4 et 4 à 5 nous avons bien $1 + 4 + 2 = 7$.

3.2 Impact des délais

Après avoir appliqué l'algorithme de Bellman-Ford à la main pour le routeur 1, les résultats obtenus sont les mêmes. Par contre nous remarquons de grand délais entre les mises à jour de la table du routeur 1 (environ 0,1s) comme l'indique les logs ici. Ceci est dû aux faibles vitesses de transmission des

liens 4-5 et 4-6. On remarque que logiquement le lien 4-6 est moins coûteux que le lien 5-6, sauf que dû à la vitesse de transmission différente des deux liens, l'information du lien 5-6 arrive en premier, celle du lien 4-6 arrive 2 millisecondes plus tard.

```
@0.013s Router 1 updated its routing table:
 2 via 2 cost 1
 3 via 3 cost 15
 4 via 2 cost 4
 5 via 3 cost 17
 6 via 3 cost 18
@0.015s Router 1 updated its routing table:
 2 via 2 cost 1
 3 via 3 cost 15
 4 via 2 cost 4
 5 via 2 cost 5
 6 via 2 cost 8
```

FIGURE 3.3 – Logs des mises à jour de la table du routeur 1.

En conclusion, le délai empêche de converger vers le chemin le moins coûteux le plus rapidement possible mais ne l'empêche pas d'arriver à la solution optimale. Ceci est logique puisque le délai est induit par le temps de transmission et de propagation, cela n'empêchera pas les informations de parvenir à destination, seulement les retarder. La capacité des liens et la taille des queues peuvent induire la perte de paquets, ce qui n'est pas considéré dans notre cas.

3.3 Comptage à l'infini

La situation modélisée est celle présentée au cours et représentée par la figure ci-dessous, constituée de 3 routeurs.

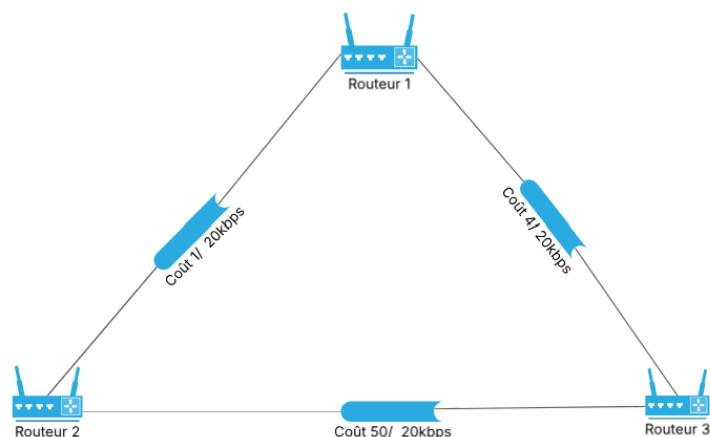


FIGURE 3.4 – Topologie du comptage à l'infini.

3.3.1 Explications

Dans la situation initiale, les meilleurs chemins pour aller jusqu'au routeur 2 est le chemin direct pour le routeur 1. Pour le routeur 3 il est plus optimal de passer par le routeur 1. Si l'on fait subitement passer le coût du chemin 1-2 à 60, le routeur 1 va vouloir recalculer la route, le routeur 3 n'étant pas encore au courant du changement, le routeur 1 va voir l'ancien coût en passant par 3 et rajouter le coût du lien 1-3 pour l'emprunter. Le routeur 1 envoie maintenant ses nouvelles données aux autres routeurs. Le coût de 3-2 étant toujours trop élevé, le routeur 3 va encore passer par le routeur 1. Cela provoque une boucle puisque le routeur 1 envoie ses paquets au routeur 3 et inversement, jusqu'à ce que l'addition des coûts devienne supérieur à celui du lien 3-2. Lors de cette boucle des paquets et du temps seront perdus.

Split Horizon et Hold-down timer

Le principe de Split Horizon consiste à ne pas renvoyer l'information à l'entité qui nous l'a transmise.

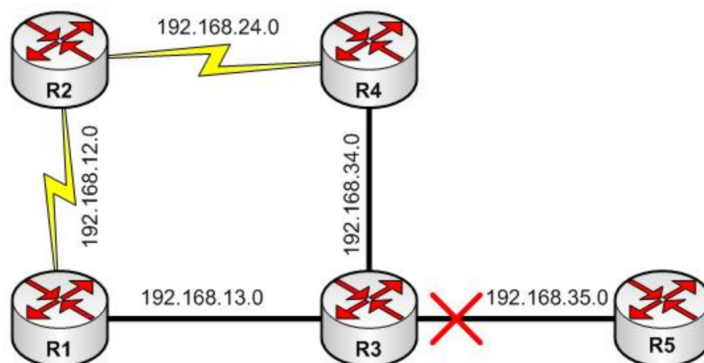


FIGURE 3.5 – Topologie d'illustration pour le Split Horizon.

Reprenons ce réseau, le routeur 3 transmet les informations sur le lien qui le lie au routeur 5, qui ne possède qu'un lien d'accès, celui avec 3. Le principe de Split Horizon consiste donc à dire que 4 et 1 ne devraient pas retransmettre l'information à 3.

Cependant le principe de Split Horizon à lui seul ne suffit pas pour éviter le phénomène de count-to-infinity. En effet, supposons que 1 et 4 soient au courant de la coupure du lien entre 3 et 5 et que malencontreusement, 2 envoie une mise à jour à 1, lui disant qu'il a un chemin vers ce lien par 4 et inversement. 1 et 4 vont donc considérer qu'ils peuvent à nouveau accéder à ce

lien et donc augmenter le coût par celui de leur lien avec 2. Cette information parvient enfin à 3 qui lui aussi considère qu'il peut réaccéder à ce lien par 1 ou 4. Il envoie donc cette mise à jour à 4, qui considère qu'il est plus avantageux de passer par 3, envoie sa mise à jour à 2, lui à 1 puis revient de nouveau à 3 et ainsi de suite...

Hold Down Timer

Ce timer est mis en place lorsque le coût d'une route est augmenté. Quand ce timer est activé, le routeur n'accepte aucune route au coût supérieur ou égal au coût actuel. Le temps de timer optimal doit être calculé en prenant en compte qu'un temps trop long rallonge la convergence du système mais qu'un temps trop court peut empoisonner ce dernier.

En conclusion, Split Horizon permet d'éviter des boucles mais est toujours sensible à certains cas particuliers, l'installation du Hold Down Timer permet de faire patienter le système le temps qu'il se stabilise.

3.3.2 Source :

`:https://www.certificationkits.com/cisco-certification/ccna-articles/cisco-ccna-distance-vector-routing-protocols-2/count-to-infin-split-horizon-rt-`